AURA — Stage 1 Formal Specification and Interface Design

Contract-defined multi-step agent task completion, evidence semantics, four JSON schemas, and acceptance cases

AURA Research Project

24 October 2026

Contents

1 AURA — Stage 1 Formal Specification and Interface Design **1**
1.1 Executive decision . . . 2
1.2 1. Sources and scope . . . 2
1.2.1 In scope . . . 2
1.2.2 Out of scope for Stage 1 . . . 2
1.3 2. Definitions and notation . . . 2
1.3.1 2.1 Verdict semantics . . . 3
1.3.2 2.2 Conditional guarantee . . . 3
1.3.3 2.3 Observation indistinguishability . . . 3
1.4 3. Contract model — `TaskContract` . . . 3
1.4.1 3.1 Example obligation . . . 4
1.4.2 3.2 `frame_policy` is not a list of expected results . . . 4
1.5 4. Probe catalogue — `ProbeSpec` . . . 4
1.5.1 4.1 Probe response constraints . . . 5
1.6 5. Evidence records — `Observation` . . . 5
1.6.1 5.1 Observation statuses . . . 5
1.6.2 5.2 Frame completeness receipts . . . 6
1.6.3 5.3 Contradictory observations . . . 6
1.7 6. Decision output — `Certificate` . . . 6
1.7.1 6.1 Top-level verdict versus diagnostics . . . 6
1.8 7. Finite reference-world model and acceptance logic . . . 7
1.8.1 7.1 Why model completeness matters . . . 7
1.9 8. Required acceptance cases . . . 7
1.9.1 8.1 Evidence-twin requirement . . . 8
1.9.2 8.2 Local process requirement . . . 8
1.10 9. Cross-object integrity rules for Stage 2 . . . 8
1.11 10. Stage 1 quality assurance . . . 9
1.11.1 Stage 1 acceptance gate . . . 9
1.12 11. Stage 2 entry contract . . . 9
1.13 12. Provenance and references . . . 10

1 AURA — Stage 1 Formal Specification and Interface Design

Project: AURA — Agent Uncertainty and Residue Arbiter
Research direction: Counterexample-Guided Evidence Certification for Multi-Step AI-Agent Task Completion
Stage: 1 of 8 implementation/research stages following Stage 0

Status: Design specification and machine-readable interface examples; no verification engine implemented
Version: 1.0, 24 October 2026

1.1 Executive decision

AURA's responsibility is to establish whether the **complete declared task** was fulfilled, including required persistent outcomes, permissible state changes, and mandatory procedural conditions. Tool invocation success and an agent's closing message do not establish completion. AURA must also recognize when decisive independent evidence is unavailable.

Four interfaces are fixed for the initial prototype: `TaskContract`, `ProbeSpec`, `Observation`, and `Certificate`. Their concrete Draft 2020-12 JSON Schemas and example instances are included in the companion schema package. The interface designs are **candidates to implement**, not experimental results or a claim of a production-ready formal guarantee.

Stage 1 acceptance: all four schemas are structurally valid; 12 finite-world reference cases agree with the expected verdicts; consistency rules are documented; and Stage 2 can implement the boundaries without guessing what inputs and outputs mean.

1.2 1. Sources and scope

This document refines the Stage 0 research-freeze document (`AURA_Research_Freeze_v1.md`) and its decision log. It preserves the previously frozen boundaries: deterministic verification, independent read-only inspections, contract-defined obligations, full accounting of declared mutation scope for clean PASS, hidden evaluation ground truth, and no ungrounded novelty promises.

1.2.1 In scope

- Reviewed contracts over SQLite records, files, trusted recorded tool events, and optional simulated external results.
- Initial and final snapshot evidence, plus complete mutation records when a task requires transient-effect checks.
- Exact counts, expected values, cross-artifact consistency, ordering, and declared side-effect frames.
- Bounded finite-world reference semantics for development tests, with conservative reasoning as the later scalable implementation.
- Read-only probes with explicit provenance, consistency epochs, failure modes, and metered costs.

1.2.2 Out of scope for Stage 1

- Code executing agent workflows, database adapters, CERA inspection planning, automatic rollback, or LLM judging.
- Claims that the contract captures unstated user intent, that observed event order proves true causal dependence, or that a snapshot covers unnamed external services.
- A probabilistic risk guarantee, full formal model checking for arbitrary live software, or production-grade adversarial evidence protection.

1.3 2. Definitions and notation

An execution world is a quadruple:

$$W = (S_0, \tau, S_T, X),$$

where `S0` is the initial state, `tau` is a trusted observation of the relevant execution history, `ST` is the terminal state, and `X` contains in-scope external effects. A reviewed contract is `C`; `C(W)` is true precisely when **every declared obligation** is satisfied.

An observation record `e` is produced by an authorized probe `a` (`ProbeSpec`) during a named consistency epoch. Let `E` be the set of accepted, independently trusted observations. Let `M` be the explicit environment model and the constraints it imposes.

$$K(E) = \{W \in M : W \text{ is consistent with } E\}.$$

For bounded reference testing, `M` is a finite set of abstract worlds plus a probe-response model. The actual simulator outcome is recorded separately by the experiment harness and **never supplied to the verifier as hidden ground truth**.

1.3.1 2.1 Verdict semantics

If `K(E)` is nonempty:

- **PASS:** all worlds in `K(E)` satisfy `C`.
- **FAIL:** all worlds in `K(E)` violate `C`.
- **UNCERTAIN:** `K(E)` contains both satisfying and violating worlds.

An empty `K(E)` is a model/evidence inconsistency. In the initial public verdict interface, it becomes **UNCERTAIN + `evidence_conflict` or `model_inconsistent`**, never a vacuously true PASS. A separate diagnostic field makes it distinguishable from ordinary lack of visibility.

A solver returning `unknown`, a timeout, or an unsupported formula is **UNCERTAIN + `solver_indeterminate`**. A heuristic failure to find a counterexample is *not* a proof that none exists.

For a solver that supports logical satisfiability, a reference rule is:

$$\begin{aligned}
&\text{SAT}(M \wedge E) \text{ must hold first;}\\
&\text{PASS} \iff \neg\,\text{SAT}(M \wedge E \wedge \neg C);\\
&\text{FAIL} \iff \neg\,\text{SAT}(M \wedge E \wedge C);\\
&\text{UNCERTAIN otherwise.}
\end{aligned}$$

The first line is essential: without it, contradictory observations could cause both success and failure queries to be unsatisfiable and incorrectly certify PASS.

1.3.2 2.2 Conditional guarantee

If the actual world is contained in `K(E)`, the contract and environment model are correct, accepted observations are genuine, and the decision procedure is sound, then **PASS implies that the actual world satisfies the declared contract**. This does not imply that the contract contains every unstated expectation in the natural-language request.

1.3.3 2.3 Observation indistinguishability

If two worlds `W1` and `W2` produce identical results for all permitted inspections but differ on `C`, no correct deterministic verifier restricted to those inspections can give decisive success/failure verdicts for both. Such evidence twins are mandatory uncertainty tests.

1.4 3. Contract model — `TaskContract`

The contract declares required outcomes, allowed changes, and the observation scope. It must be authored or approved independently from the agent execution, versioned, and hashed before evaluation.

Required top-level fields

Field	Meaning
`schema_version`	Interface version, initially `1.0`
`contract_id` / `contract_version`	Stable task contract identity
`task_description`	Human-readable explanation, not executable proof logic
`surfaces`	Typed database, filesystem, trace, or mock-external boundaries
`frame_policy`	Permitted mutations and which mutable surfaces require full coverage
`obligations`	Independent, machine-checkable predicates with stable IDs

Each obligation has a `category` (`outcome`, `frame`, `process`, `external`), `surface_id`, and a restricted predicate. The v1 predicate operators are: `eq`, `neq`, `true`, `false`, `count_eq`, and `same_as`. **No arbitrary SQL or free-text criterion is accepted inside a predicate.** Adapters may generate facts, but they do not redefine contract truth.

`required_complete_coverage` lists the mutable surface IDs for which a coverage witness is required. An obligation such as "no other student was modified" must not PASS from a query of only the expected student row.

1.4.1 3.1 Example obligation

```
{
  "id": "exact_record_count",
  "description": "Exactly one matching student row exists",
  "category": "outcome",
  "surface_id": "sqlite.students",
  "predicate": {
    "operator": "count_eq",
    "fact_key": "student.101.matching_count",
    "expected": 1
  },
  "required_complete_coverage": []
}
```

This predicate prevents existential checks from accepting two otherwise matching student records.

1.4.2 3.2 `frame_policy` is not a list of expected results

Required outcomes and permitted mutations are separate. The task may require a marks update and allow a related audit entry without requiring that the audit entry exist.

The v1 `frame_policy` defines the declared mutable surfaces, allowed effects, and the required method of evidence accounting (`snapshot_pair`, `complete_journal`, `either`, or `none`). In `closed_world` mode, every mutated entity within the declared surface must be explained by an allowed-effect rule. Known permitted noise (such as selected timestamps) must be declared narrowly and tested, not ignored wholesale.

Key limitation: A before/after diff verifies persistent final-state differences. It cannot establish that an unrelated row was *never temporarily changed and reverted*. A separate fully mediated mutation record is required for transient-effect obligations.

1.5 4. Probe catalogue — `ProbeSpec`

A probe is an authorized **read-only** method for collecting evidence. It is distinct from the agent's task-changing tools.

Required fields: `probe_id`, `surface_id`, `adapter`, `read_only`, `independence`, `provides_fact_keys`, `mutation_coverage`, `covered_mutation_surfaces`, `coverage_method`, `estimated_cost`, and `failure_modes`.

Probe feature	Semantics
`read_only`	Must be `true`; real adapter must enforce no user-state change
`independence`	`independent_verifier`, `conditionally_trusted`, or `agent_self_report`
`provides_fact_keys`	Facts it is authorized to assert or refute
`mutation_coverage`	`none`, `scoped`, or `complete`
`covered_mutation_surfaces`	Surfaces covered by the asserted mutation accounting
`coverage_method`	None, quiescent initial/final snapshot pair, or complete mutation journal
`estimated_cost`	Planning estimate; distinct from measured actual cost
`failure_modes`	Declared timeout, denial, stale evidence, integrity failures, etc.

A probe may claim `complete` mutation coverage only when the underlying collection mechanism has an independently justified completeness guarantee over precisely named surfaces and a consistent observation epoch. Merely returning `count=0` is insufficient.

1.5.1 4.1 Probe response constraints

- No probe may emit facts outside its registered `provides_fact_keys`.
- `agent_self_report` is not authoritative for persistence or complete mutation coverage.
- Probe result status `unavailable` or `error` never silently implies a requested effect happened or did not happen.
- Probe cost must include failed inspections and preliminary snapshots or journals where necessary.
- A provider timestamp and a logical epoch ID are not, alone, an atomicity guarantee. The adapter must substantiate a consistent snapshot or an explicit freshness bound.

1.6 5. Evidence records — `Observation`

An observation is the immutable evidence record for one probe execution, not the task result itself.

Required fields: `observation_id`, `run_id`, `probe_id`, `epoch_id`, `observed_at`, `status`, `reason_code`, `channel`, `facts`, `coverage_receipts`, `actual_cost`, and `payload_sha256`.

1.6.1 5.1 Observation statuses

Status	Interpretation
`ok`	Probe completed; its facts may become eligible evidence after trust/epoch validation
`unavailable`	The requested state was not observed; the absence is not a negative fact
`error`	The inspection failed or has an integrity issue; the failure is not task failure by itself

Examples of `reason_code` include `timeout_no_readback`, `permission_denied`, `stale`, and `integrity_failure`. `none` is reserved for successful observations without an error condition.

1.6.2 5.2 Frame completeness receipts

An `Observation` can carry a `coverage_receipt` with the mutable surface ID, completeness grade, method, consistent-epoch declaration, source independence, and before/after artifact references when applicable.

A receipt claiming that the declared `sqlite.students` table and `filesystem.reports` tree were fully compared is not equivalent to a receipt for every external service. The scope matters.

A SHA-256 digest provides a reproducible integrity reference; it does **not** authenticate an untrusted source by itself. Trust is granted through the observation boundary and verifier isolation assumptions.

1.6.3 5.3 Contradictory observations

Two independent observations asserting `student.101.marks = 85` and `student.101.marks = 90` at the same logically consistent epoch cannot both be accepted as one coherent snapshot of that fact. The system must flag a contradiction; it cannot select the favorable value or treat the empty consistent-world set as successful verification.

Before declaring a conflict, implementations must check whether observations are actually about the same entity, same field, and same consistency epoch. Changed values across different epochs are not inherently contradictory.

1.7 6. Decision output — `Certificate`

A certificate is AURA's auditable claim about a **single run under a single contract version and observation model**.

Required fields: `certificate_id`, `run_id`, `contract_id`, `contract_sha256`, `epoch_id`, `mode`, `verdict`, `decision_status`, `reason_codes`, `obligation_results`, `inspection_ids`, `evidence_refs`, `verified_frame_surfaces`, `unresolved_obligation_ids`, `costs`, and `trust_assumptions`.

Field	Requirement
`mode`	`bounded_exact` or `conservative`; never mislabel a heuristic as exact
`verdict`	`PASS`, `FAIL`, or `UNCERTAIN`
`decision_status`	`normal`, `evidence_conflict`, `model_inconsistent`, or `solver_indeterminate`
`obligation_results`	Every contract obligation appears with `pass`, `fail`, or `unknown`
`evidence_refs`	Stable links to independently collected observation records
`unresolved_obligation_ids`	Identifies the basis for abstention
`costs`	Capture, inspection, and planner latency plus bytes read; sum without hiding preprocessing
`trust_assumptions`	Explicit limits that qualify the decision

A certificate is **not an agent-generated explanation**, and it is not an unconditional proof of user-intent satisfaction. In the bounded-exact research mode, it should be replayable from the declared model and the accepted observations. It must contain no harness-only actual-world labels, injected-fault IDs, or benchmark answer keys.

1.7.1 6.1 Top-level verdict versus diagnostics

Keep only three public verdicts. Use structured reason codes, for example:

- `required_report_absent`, `record_multiplicity_violation`, `out_of_scope_mutation`.
- `unconfirmed_external_effect`, `frame_coverage_incomplete`, `transient_mutation_unobserved`.
- `evidence_conflict`, `model_inconsistent`, `solver_indeterminate`.

If an observed required property is definitively false and another is unknown, the global decision is **FAIL** (not UNCERTAIN), assuming the false finding is genuinely supported.

1.8 7. Finite reference-world model and acceptance logic

For Stage 1, the formal reference model is intentionally finite. Each example has:

1. A complete `task_contract.json`.
2. A compatible `probe_catalogue.json`.
3. One or more `observations.json` records.
4. A verifier-visible `reference_possible_worlds.json` catalogue of candidate abstract states.
5. An `expected.json` verdict and diagnostic.
6. A separately stored **harness-only** actual-world label.

The reference worlds contain abstract facts such as `student.101.marks`, `report.101.exists`, and `mutation.out_of_frame_co` They do not model SQLite's full transaction semantics. Every successful reference-world test must include plausible counterfactual worlds so that certification does not follow trivially from a singleton world model.

Compatibility rule: a world is compatible with an accepted authoritative observation only if its fact assignments match the observation and the declared probe-response semantics. Unavailable observations supply no asserted state fact. Model-incomplete or inconsistent probe semantics must cause abstention or a model-integrity diagnostic.

Frame rule: a clean PASS requires a complete consistent coverage witness for each declared mutable frame surface (and, when explicitly required, trusted intermediate mutation accounting). Even if the abstract possible-world catalogue accidentally contains only "clean" worlds, missing mandatory frame coverage cannot justify a clean PASS.

Truth evaluation: a predicate may be confirmed through an authorized read-back; an agent's statement that the predicate is true does not turn it into a trusted observation.

1.8.1 7.1 Why model completeness matters

A finite reference model is useful only if it contains the failure alternatives relevant to the specification. The framework must not construct a candidate-world set by reading a hidden actual-world label and discarding all adverse possibilities. A production verifier would instead generate a sound over-approximation of feasible states or abstain where no sound model is available.

The included `reference_possible_worlds.json` instances are **educational model fixtures, not evidence of real-world model completeness**. They are designed to test the truth rules, not to demonstrate AURA's effectiveness on unrestricted agent executions.

1.9 8. Required acceptance cases

The 12 author-validated examples are stored under `examples/`. The table gives the required response from the **future engine**, not a measured result from an implemented engine.

Case	Condition	Required decision
01	All outcomes, counts, process rules, and frame receipts established	**PASS**
02	Report absent despite successful database update	**FAIL**
03	Unauthorized unrelated mutation observed	**FAIL**
04	Duplicate record violates exact multiplicity	**FAIL**
05	External notification outcome unobservable; both worlds possible	**UNCERTAIN**
06	Wrong marks observed while notification remains unknown	**FAIL**

Case	Condition	Required decision
07	Target checks succeed but full frame coverage unavailable	**UNCERTAIN**
08	Two trusted same-epoch observations disagree	**UNCERTAIN + evidence_conflict**
09	Mandatory validation missing despite correct final files	**FAIL**
10	Final diff clean but prohibited transient effect cannot be checked	**UNCERTAIN**
11	Different but permitted tool path and all obligations met	**PASS**
12	Unobserved external mutable surface in declared frame	**UNCERTAIN**

These tests distinguish **factual failure** from **lack of evidence**. They also ensure frame completeness, process conditions, and inconsistent evidence cannot be ignored.

1.9.1 8.1 Evidence-twin requirement

Case 05 includes two candidate worlds that differ in actual external notification delivery but produce the same observations because the status readback is unavailable. The verifier must return UNCERTAIN. The hidden experiment harness may privately know which world actually happened; AURA must not.

1.9.2 8.2 Local process requirement

Case 09 has a correct-looking terminal state but an explicitly required validation step is absent from the trusted trace. AURA should FAIL for the procedural contract violation. This does not mean AURA must check every agent action; it checks only the process clauses **declared by the task**.

1.10 9. Cross-object integrity rules for Stage 2

The supplied Draft 2020-12 JSON Schemas validate types and shapes. The following semantic constraints deliberately require additional application-level validation later:

Identity and registration. Obligation IDs, probe IDs, and observation IDs must be unique within their respective scopes. Every referenced `surface_id`, `probe_id`, and `fact_key` must exist in its registered catalogue. A probe may assert only facts in its `provides_fact_keys`.

Predicate typing. `eq`, `neq`, and `count_eq` require `expected` (`count_eq` must use a nonnegative integer). `true/false` do not take an `expected` value. `same_as` requires `comparison_fact_key`. Unknown operators and arbitrary executable expressions are rejected.

Epoch discipline. Snapshot pairs used for one decision must correspond to a consistent, identified execution epoch. Cross-epoch evidence must be rejected for a same-state assertion unless a declared reconciliation rule justifies it.

Trust and independence. A self-report probe cannot authenticate its own effect; complete-frame assertions require independent, integrity-checked evidence. A certificate's evidence references must resolve to existing accepted observations.

Frame completeness. A `closed_world` contract cannot get a clean PASS unless every declared mutable surface has suitable completeness coverage. Partial table reads do not count. Final snapshots cannot satisfy a separately declared transient-history requirement.

No hidden truth. The verifier package may know abstract possible states and probe-response models, but must not import `harness_hidden/` actual outcomes or fault IDs.

Cost accounting. Measured costs are nonnegative and include failed probes and mandatory snapshot acquisition. Planner time is separately measured. Cost estimates are never silently substituted for measured cost in final evaluation.

Certificate consistency. Contract hashes must match canonical bytes; obligation result IDs must match the contract; `PASS` with unresolved obligations or missing mandatory coverage is prohibited; an evidence-integrity conflict always prevents PASS.

Versioning. Schema changes require a new `schema_version`; changes to evaluated contract content require a different canonical contract hash and may invalidate comparisons to earlier runs.

1.11 10. Stage 1 quality assurance

The companion package includes four schemas, 12 complete reference fixtures, an illustrative success certificate, a CSV test inventory, a separate hidden-world harness directory, and an offline authoring-validation report.

The authoring checks verify:

- Valid Draft 2020-12 schemas.
- All example contracts, probes and observations structurally validate.
- An illustrative `Certificate` validates.
- All 12 fixture decisions match a bounded reference implementation used only to **check the design examples**.

What has not been done: no AURA verification engine exists yet; the CERA planner has not been implemented; there are no experimental results, runtime benchmarks, external integrations, or paper findings. These authoring validations are preliminary specification QA only.

1.11.1 Stage 1 acceptance gate

- ☒ Four schemas drafted and structural validation completed.
- ☒ Twelve edge-case reference fixtures drafted and evaluated by offline specification QA.
- ☒ Bounded-world verdict rules and evidence-conflict handling documented.
- ☒ Frame completeness and hidden-ground-truth restrictions recorded.
- ☒ Data interfaces and future semantic validation requirements documented.
- ☐ Independent research-team or advisor review of the contract semantics.
- ☐ Run the same cases against an independently implemented Stage 3 verification engine.

The first five items constitute completed *pre-implementation specification work*; the last two are explicit review/implementation gates, not blockers to beginning Stage 2 engineering.

1.12 11. Stage 2 entry contract

Next, build the **Trusted Environment & Evidence Infrastructure** without implementing planner intelligence yet.

Implementation order:

1. Create an isolated sandbox with a deterministic SQLite schema and file workspace.
2. Implement quiescent initial/final snapshots and canonical state differences.
3. Implement four read-only probe adapters consistent with the schemas.
4. Persist immutable observation records, coverage receipts, consistent-epoch metadata, and actual costs.
5. Validate cross-object semantics described above.
6. Demonstrate that injected local modifications are reflected in the observation records, **without using hidden truth to supply verifier facts**.
7. Run integration tests for incomplete coverage, stale snapshots, readback denial, and contradictory evidence.

Stage 2 exit criterion: trustworthy raw evidence collection exists independently of the agent's self-report, the four schemas have functioning semantic validation, and every declared snapshot/coverage guarantee can be tested. Only then proceed to Stage 3's completion-verdict engine.

1.13 12. Provenance and references

This is a formal-design artifact derived from the team's Stage 0 research freeze and accompanying 33-page independent review. It does **not** introduce fresh empirical literature claims. Relevant methodological references in the Stage 0 comparison include:

- AppWorld (2024): https://arxiv.org/abs/2407.18901
- Agent-Diff (2026): https://arxiv.org/abs/2602.11224
- AgentLTL (2026): https://arxiv.org/abs/2607.02599
- EffectMatch (2026): https://arxiv.org/abs/2609.31301
- *Can We Trust 'Done'?* (2026): https://agentaction.dev/research/completion-assessment
- BAVAR (2026 preprint): https://www.preprints.org/manuscript/202608.2057
- Interlock (2026): https://arxiv.org/abs/2610.19686

Any publication novelty or method-comparison assertion must be grounded in a separate full-method literature audit and actual experimental results.